

SQL Queries

 <https://github.com/learn-co-curriculum/python-p3-sql-queries>  <https://github.com/learn-co-curriculum/python-p3-sql-queries/issues/new>

Learning Goals

- Use SQL to store data and retrieve it later on.
 - Use SQLite to build relational databases on your computer.
 - Perform CRUD operations on relational databases using SQL.
-

Key Vocab

- **SQL (Structured Query Language)**: a programming language that is used to manage relational databases and perform operations on the data that they contain.
 - **Relational Database**: a collection of data that is organized in well-defined relationships. The most common type of database.
 - **Query**: a statement used to return data from a database.
 - **Table**: a collection of related data in a database. Composed of rows and columns. Similar to a class in Python.
 - **Row**: a single record in a database table. Each column represents an attribute of the record. Similar to an object in Python.
 - **Column**: a single field in a database table. Each row contains values in each column. Similar to a Python object's attributes.
 - **Schema**: a blueprint of the construction of the tables in a database and how they relate to one another.
-

Introduction

In this lesson, we'll cover how to write SQL queries to retrieve and add specific data to SQL database tables.

[Fork and clone this lesson](https://github.com/learn-co-curriculum/phase-3-sql-queries/fork)  <https://github.com/learn-co-curriculum/phase-3-sql-queries/fork> to follow along.

What is a SQL Query?

The term "query" refers to any SQL statement that **retrieves** data from your database. In fact, we've already written a number of SQL queries using basic **SELECT** statements. We've already seen how to retrieve single units of data, or rows, with queries like these:

To select all of the rows from a **cats** table:

```
SELECT * FROM cats;
```

To select only rows representing data meeting certain conditions:

```
SELECT * FROM cats WHERE name = "Maru";
```

What if, however, we wanted to select the oldest cat? Or all of the cats that don't currently belong to an owner? Or all of the cats with short names?

Data storage isn't very useful if we can't **manipulate**, **view**, and **analyze** that data. Luckily for us, SQL is actually a powerful tool for doing just that.

In this exercise, we'll walk through executing a handful of common and handy SQL queries.

► *Is a **DELETE** statement a query?*

Code Along: SQL Queries

Creating our Database

For this code along we have **pets_database.db** file with a **cats** table.

Just like with the last code along, you can run the SQL commands one of two ways, depending on your preference. You can either open the database using the `sqlite3` CLI, and run the SQL commands from the terminal:

```
$ sqlite3 pets_database.db
```

Or you can open the `pets_database.db` file in DB Browser for SQLite, and run the SQL commands from the "Execute SQL" tab.

Let's check out our `cats` table now:

```
SELECT * FROM cats;
```

This should return:

```
1|Maru|3|Scottish Fold|1
2|Hana|1|Tabby|1
3|Lil\' Bub|5|American Shorthair|1
4|Moel|0|Tabby|1
5|Patches|2|Calico|1
```

Top-Tip: In `sqlite3`, you can format the output of your select statements with a few helpful options:

```
.headers on      # output the name of each column
.mode column     # now we are in column mode, enabling us to run the next two .width commands
.width auto      # adjusts and normalizes column width
# or
.width NUM1, NUM2 # customize column width
```

Run the first two commands from the `sqlite3` prompt and then execute the above `SELECT` statement again and you should see something like this:

id	name	age	breed	owner_id
1	Maru	3	Scottish Fold	1

2	Hana	1	Tabby	1
3	Lil\' Bub	5	American Shor	
4	Moe	10	Tabby	
5	Patches	2	Calico	

Much better.

ORDER BY

The first query modifier we'll explore is **ORDER BY**. This modifier allows us to order the table rows returned by a certain **SELECT** statement. Here's a boilerplate **SELECT** statement that uses **ORDER BY**:

```
SELECT column_name FROM table_name ORDER BY column_name ASC|DESC;
```

Let's select our cats and order them by age:

```
SELECT * FROM cats ORDER BY age;
```

This should return the following:

id	name	age	breed	owner_id
2	Hana	1	Tabby	1
5	Patches	2	Calico	
1	Maru	3	Scottish F	1
3	Lil\' Bub	5	American S	
4	Moe	10	Tabby	

When using **ORDER BY**, the default is to order in ascending order. If we want to specify though, we can use **ASC** for "ascending" or **DESC** for "descending." Let's try to select all of our cats and sort them by age in descending order.

```
SELECT * FROM cats ORDER BY age DESC;
```

This should return

id	name	age	breed	owner_id
4	Moe	10	Tabby	
3	Lil\ ' Bub	5	American S	
1	Maru	3	Scottish F	1
5	Patches	2	Calico	
2	Hana	1	Tabby	1

LIMIT

What if we want the oldest cat? If we want to select extremes from a database table — for example, the employee with the highest paycheck or the patient with the most recent appointment — we can use **ORDER BY** in conjunction with **LIMIT**.

LIMIT is used to determine the number of records you want to return from a dataset. For example:

```
SELECT * FROM cats ORDER BY age DESC LIMIT 1;
```

This part of the statement: **SELECT * FROM cats ORDER BY age DESC** returns all of the cats in order from oldest to youngest. Setting a **LIMIT** of **1** returns just the first, i.e. oldest, cat on the list.

Execute the above statement in your terminal and you should see:

id	name	age	breed	owner_id
4	Moe	10	Tabby	

Let's get the two oldest cats:

```
SELECT * FROM cats ORDER BY age DESC LIMIT 2;
```

Execute that statement and you should see:

id	name	age	breed	owner_id
-----	-----	-----	-----	-----
4	Moe	10	Tabby	
3	Lil\ ' Bub	5	American S	

► *How could you retrieve the three youngest cats?*

BETWEEN

As we've already established, being able to sort and select specific data sets is important. Continuing on with our example, let's say we urgently need to select all of the cats whose age is between 1 and 3. To create such a query, we can use **BETWEEN**. Here's a boilerplate **SELECT** statement using **BETWEEN**:

```
SELECT column_name(s) FROM table_name WHERE column_name BETWEEN value1 AND value2;
```

Let's try it out on our **cats** table:

```
SELECT name FROM cats WHERE age BETWEEN 1 AND 3;
```

This should return:

```
Maru
Hana
Patches
```

NULL

Let's say the administrator of our Pets Database has found a new cat. This kitty doesn't have a name yet, but should be added to our database right away. We can add data with missing values using the **NULL** keyword.

Let's insert our new cat into the database. Our abandoned kitty has a breed, but no name or age as of yet:

```
INSERT INTO cats (name, age, breed) VALUES (NULL, NULL, "Tabby");
```

Now, if we look at our `cats` data with `SELECT * FROM cats;`, we should see:

id	name	age	breed	owner_id
1	Maru	3	Scottish Fold	1
2	Hana	1	Tabby	1
3	Lil\ ' Bub	5	American Shor	
4	Moe	10	Tabby	
5	Patches	2	Calico	
6			Tabby	

We can even select the mysterious, nameless kitty with the following query:

```
SELECT * FROM cats WHERE name IS NULL;
```

This should return the following:

id	name	age	breed	owner_id
6			Tabby	

COUNT

Now, we'll talk about a SQL aggregate function, `COUNT`.

SQL aggregate functions are SQL statements that operate on groups of records in our database rather than individual records. For example, we can retrieve minimum and maximum values from a column, sum values in a column, get the average of a column's values, or count a number

of records that meet certain conditions. You can learn more about these SQL aggregators [here](#)  (<http://www.sqlclauses.com/sql+aggregate+functions>) and [here](#)  (<http://zetcode.com/db/sqlite/select/>).

For now, we'll just focus on **COUNT**. **COUNT** will count the number of records that meet a certain condition. Here's a boilerplate SQL query using **COUNT**:

```
SELECT COUNT([column name]) FROM [table name] WHERE [column name] = [value]
```

Let's try it out and count the number of cats who have an **owner_id** of **1**:

```
SELECT COUNT(owner_id) FROM cats WHERE owner_id = 1;
```

This should return:

```
COUNT(owner_id)
-----
2
```

GROUP BY

Lastly, we'll talk about the handy aggregate function **GROUP BY**. Like its name suggests, it groups your results by a given column.

Let's take our table of cats

id	name	age	breed	owner_id
-----	-----	-----	-----	-----
1	Maru	3	Scottish Fold	1
2	Hana	1	Tabby	1
3	Lil\ ' Bub	5	American Shor	
4	Moe	10	Tabby	
5	Patches	2	Calico	
6			Tabby	

Here, we can see at a glance that there are three tabby cats and one of every other breed — but what if we had a larger database where we couldn't easily see the number of cats *grouped by* breed? That's where — you guessed it! — **GROUP BY** comes in handy.

```
SELECT breed, COUNT(breed) FROM cats GROUP BY breed;
```

This should return

breed	COUNT(breed)
-----	-----
American Shorthair	1
Calico	1
Scottish Fold	1
Tabby	3

GROUP BY is a great function for aggregating results into different segments — you can even use it on multiple columns!

```
SELECT breed, owner_id, COUNT(breed) FROM cats GROUP BY breed, owner_id;
```

Below you can see that the cats are still grouped by breed but are also now further broken down by owner_id, so the two tabby cats without an owner_id are listed separately from the tabby who belongs to owner 1.

breed	owner_id	COUNT(breed)
-----	-----	-----
American Shorthair		1
Calico		1
Scottish Fold	1	1
Tabby		2
Tabby	1	1

Note on **SELECT**

We are now familiar with this syntax:

```
SELECT name FROM cats;
```

However, you may not know that this can be written like this as well:

```
SELECT cats.name FROM cats;
```

Both return:

```
name
-----
Maru
Hana
Lil\' Bub
Moe
Patches
```

SQLite allows us to explicitly state the `tableName.columnName` we want to select. This is particularly useful when we want data from two different tables.

Imagine we have another table called `dogs` with a column for the dog names:

```
CREATE TABLE dogs (
  id INTEGER PRIMARY KEY,
  name TEXT
);
```

```
INSERT INTO dogs (name) VALUES ("Clifford");
```

If we want to get the names of all the dogs and cats, we can no longer run a query with just the column name. `SELECT name FROM cats,dogs;` will return `Error: ambiguous column name: name`.

Instead, we must explicitly follow the `tableName.columnName` syntax.

```
SELECT cats.name, dogs.name FROM cats, dogs;
```

You may see this in the future. Don't let it trip you up.

Conclusion

SQL gives you a lot of tools for fine-grained control over how to view data from various database tables. When you start working with larger databases that have 5000 or 50,000 rows in a table instead of 5, having this level of control is crucial for accessing and analyzing data that's useful to your applications, and can help you improve your application's performance significantly by limiting the amount of data being returned.

Resources

- [SQL Tutorial - W3Schools](https://www.w3schools.com/sql/) ➞ [\(https://www.w3schools.com/sql/\)](https://www.w3schools.com/sql/)
- [Documentation - SQLite](https://www.sqlite.org/docs.html) ➞ [\(https://www.sqlite.org/docs.html\)](https://www.sqlite.org/docs.html)
- [SQLite - VisualStudio Marketplace](https://marketplace.visualstudio.com/items?itemName=alexcvzz.vscode-sqlite) ➞ [\(https://marketplace.visualstudio.com/items?itemName=alexcvzz.vscode-sqlite\)](https://marketplace.visualstudio.com/items?itemName=alexcvzz.vscode-sqlite)
- [Datatypes in SQLite - SQLite](http://www.sqlite.org/datatype3.html) ➞ [\(http://www.sqlite.org/datatype3.html\)](http://www.sqlite.org/datatype3.html)
- [SQLite Keywords - SQLite](https://www.sqlite.org/lang_keywords.html) ➞ [\(https://www.sqlite.org/lang_keywords.html\)](https://www.sqlite.org/lang_keywords.html)
- [ZetCode sqlite3 Tutorial](http://zetcode.com/db/sqlite/) ➞ [\(http://zetcode.com/db/sqlite/\)](http://zetcode.com/db/sqlite/)